

PDV Connect — Guide de soutenance technique

Document de préparation oral : architecture, BDD, sécurité, scaling, RBAC/ABAC, Q&R jury.

1. Présentation en 30 secondes

PDV Connect est une plateforme de gestion de réseaux d'agents Mobile Money (Togo : YAS/Mixx, FLOOZ/Moov). Elle centralise les transactions, les kiosques, la trésorerie et la paie.

Contexte : produit **commandé par une entreprise cliente**, développé en méthode **Agile**, **déjà livré** et **déployé en production** (VPS + app Android sur le terrain).

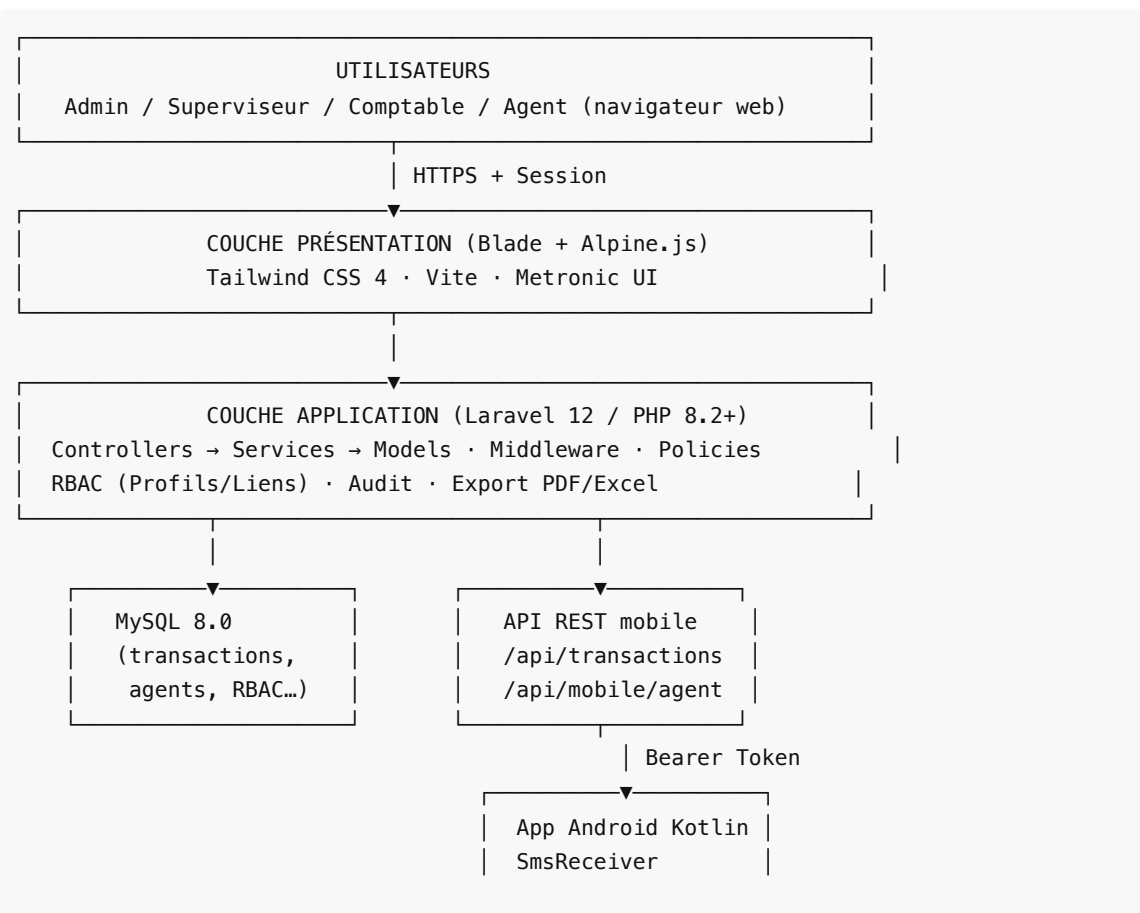
Différenciateur : une application Android native (Kotlin) intercepte automatiquement les SMS opérateurs et les envoie à l'API Laravel.

Stack : Laravel 12 + MySQL 8 + Blade/Tailwind + app Android Kotlin + Docker.

Phrase contexte entreprise (15 sec)

« PDV Connect n'est pas un prototype académique : c'est une **solution commandée par une entreprise**, livrée, installée sur leurs agents terrain, et utilisée pour piloter leur réseau Mobile Money au quotidien. »

2. Architecture globale



Room + WorkManager

Déploiement (2 VPS)

VPS	Rôle
VPS 1	Laravel + MySQL + scheduler (sauvegardes quotidiennes)
VPS 2	MinIO (stockage S3 des backups)

Fichiers : `docker-compose.vps.yml` , `docker-compose.minio.yml`

2bis. Architecture logicielle et méthode d'analyse

Architecture utilisée

Le projet combine **plusieurs modèles** selon la couche :

1. Architecture globale — Client/Serveur en 3 tiers (+ mobile)

Tier	Composant	Techno
Présentation	Interface web + app Android	Blade/Alpine.js · Kotlin
Métier / Application	Logique business, API, sécurité	Laravel (Controllers, Services)
Données	Persistance	MySQL 8 · Room (SQLite mobile)

```
[Client web]  —HTTPS→ [Serveur Laravel] → [MySQL]
[Client Android] —REST→      ↑
                        [API /api/...]
```

2. Backend web — MVC (Model-View-Controller) — pattern Laravel

Couche MVC	Dans PDV Connect
Model	Eloquent (Agent, Transaction, Utilisateur...)
View	Templates Blade (resources/views/)
Controller	TransactionController, AgentController...

Couche Service en plus (bonne pratique Laravel) :

- `RoutePermissionService` — RBAC
- `IpBlockService` — sécurité
- `AgentPhoneResolver` — résolution agent

Middleware = filtres transversaux (auth, permissions, token SMS, IP bloquée).

3. API mobile — REST

- Endpoints JSON stateless (sauf token Bearer)
- `POST /api/transactions/from-sms`

- `POST /api/mobile/agent/login`

4. App Android — Architecture en couches

Couche	Packages	Rôle
UI	ui/	MainActivity, écrans agent
SMS / Métier	sms/	SmsReceiver, SmsParser
Réseau	api/	Retrofit, ApiClient
Sync	sync/	OfflineSyncRepository, WorkManager
Données locales	data/local/	Room, DataStore
Utilitaires	util/	Réseau, notifications, SIM

Pattern **offline-first** : données locales d'abord, sync ensuite.

5. Déploiement — Conteneurisée (Docker)

- Monolithe Laravel containerisé (pas microservices)
- Services séparés : `app` , `mysql` , `scheduler` , `MinIO`
- = architecture **modulaire monolithique** prête à scaler

Phrase soutenance architecture (30 sec)

« PDV Connect suit une architecture **client/serveur en 3 tiers** : présentation web et mobile, couche métier Laravel en **MVC**, base MySQL. L'app Android est **en couches** avec une approche **offline-first**. L'API REST fait le lien entre mobile et serveur. Le tout est **containerisé avec Docker** sur deux VPS. »

Méthode d'analyse et de développement — Agile (Scrum)

Méthode retenue : Agile (Scrum) — sur **commande entreprise**, avec livraisons successives jusqu'à la mise en production.

Contexte projet

Élément	PDV Connect
Commanditaire	Une entreprise cliente (réseau d'agents Mobile Money)
Nature	Produit sur mesure répondant à un cahier des charges métier
Statut	Livré et en production (plateforme web + APK agents)
Méthode	Agile — sprints, retours client, itérations jusqu'à la version stable (APK v2.9)

Pourquoi Agile pour PDV Connect ?

- **Cahier des besoins évolutif** de l'entreprise cliente (formats SMS YAS/FLOOZ affinés en tests terrain)
- **Livraisons fréquentes** au client (APK v1.4 → v2.9, modules web par sprint)
- **Retours client / agents terrain** intégrés à chaque itération (parsing, sync, identification agent)
- **Mise en production progressive** : web → API → mobile → fiabilité → sécurité / VPS

Termes Agile — vocabulaire et application PDV Connect

Terme	Chez toi (PDV Connect)
Backlog	Liste des fonctionnalités demandées par l'entreprise : transactions, mobile SMS, RBAC, trésorerie, rapports...
Sprint	Itération de dev (ex. Sprint mobile = app Android + API SMS)
User story	« En tant qu'agent, je veux que mes SMS partent automatiquement vers la plateforme »
Incrément	Version livrable remise au client à chaque sprint (ex. APK v2.x, module web opérationnel)
Product Owner	L' entreprise cliente (priorise le backlog, valide les livraisons)
Livraison finale	Plateforme déployée sur VPS + APK distribuée aux agents (/app-mobile)

Comment Agile a été appliqué

Concept Agile	Application PDV Connect
Product Backlog	Liste des fonctionnalités : transactions, agents, RBAC, mobile SMS, trésorerie, rapports...
Sprints	Itérations de dev (ex. Sprint 1 : auth + dashboard, Sprint 2 : transactions, Sprint 3 : app Android...)
User Stories	« En tant qu'agent, je veux que mes SMS soient envoyés auto à la plateforme »
Incrément	Version livrable à chaque sprint (web fonctionnel → API SMS → APK testable)
Tests continus	SmsParserTest, AuthenticationSecurityTest à chaque itération
Rétrospective	Corrections après retours terrain (ex. multipart SMS, file offline, agent phone)

Sprints réalisés (commande entreprise)

Sprint	Objectif	Livrable remis au client
Sprint 1	Fondations web	Auth, RBAC, CRUD agents/kiosques
Sprint 2	Transactions & soldes	Saisie manuelle, dashboard KPIs
Sprint 3	API + app mobile	Capture SMS, envoi API
Sprint 4	Fiabilité mobile	Room, WorkManager, parsing robuste
Sprint 5	Entreprise & production	Trésorerie, paie, audit, déploiement Docker VPS

UML dans une approche Agile

L'UML n'est **pas incompatible** avec Agile : elle sert à **clarifier** le sprint en cours (cas d'utilisation, séquence SMS, classes métier), sans tout figer dès le départ.

Diagramme	Quand
Cas d'utilisation	Sprint 0 / backlog initial
Classes	Avant ou pendant chaque module
Séquence SMS	Sprint mobile
Déploiement	Sprint infra Docker

Phrase soutenance méthode (30 sec)

« L'**entreprise cliente** nous a commandé PDV Connect. Nous avons travaillé en **Agile** : backlog validé avec le client, **sprints** avec livraisons incrémentales, retours terrain intégrés à chaque version. Le produit est **livré** — plateforme web en production sur VPS, APK v2.9 chez les agents. »

Si le jury demande « Cycle en V ou Agile ? »

« **Agile**. La cliente ne voulait pas attendre la fin pour tester : chaque sprint livrait une version utilisable. Les retours des agents sur le terrain — formats SMS, réseau, identification — ont alimenté les sprints suivants jusqu'à la livraison finale. »

Si le jury demande « C'est quoi Agile ? »

« Développer par **petites itérations** appelées sprints. À chaque sprint : on choisit des fonctionnalités dans le backlog, on les développe, on teste, on livre. Si le besoin change — comme les formats SMS opérateurs — on s'adapte au sprint suivant, sans repartir de zéro. »

3. Base de données — schéma métier

Tables principales

Table	Rôle
utilisateurs	Comptes web (email, mot de passe hashé, statut)
profils	Rôles RBAC (Super Admin → Agent, avec niveau 0–3)
liens	Menus / routes de l'application
profil_liens	Matrice permissions (profil ↔ lien), soft delete
user_profils	Affectation utilisateur ↔ profil
agents	Agents terrain, liés à un kiosque
kiosques	Points de vente (géolocalisation)
transactions	Opérations Mobile Money / agence / SMS auto
opérateurs	YAS, FLOOZ
soldes	Soldes espèce/virtuel par agent et opérateur
salaires / parametres_salaire	Paie (fixe, commission, mixte)

mouvements_tresorerie	Flux trésorerie entreprise
config_app_mobile	Token API SMS, filtres opérateurs
system_logs	Journal d'audit
blocked_ips	Liste noire IP
agent_kiosque_historique	Historique affectations

Relations clés

```

utilisateurs —N:N— profils (user_profils)
profils —N:N— liens (profil_liens)          ← RBAC
utilisateurs —1:1— agents (optionnel)
agents —N:1— kiosques
agents —1:N— transactions, soldes, salaires
transactions —N:1— operateurs

```

Patterns techniques BDD

- **UUID** (uid) sur entités métier (Utilisateur, Agent, Transaction)
- **Soft deletes** sur pivots RBAC et entités sensibles
- **Migrations incrémentales** datées + seeders data-as-code
- **Charset** : utf8mb4 (support caractères spéciaux, emojis)

4. Pourquoi ces technologies ?

Laravel (PHP)

Critère	Justification
Maturité	Framework éprouvé, écosystème riche
Rapidité dev	ORM Eloquent, migrations, auth intégrée
Sécurité	Middleware, CSRF, hashing bcrypt natif
Écosystème Togo/Afrique	Large adoption, facilité recrutement/maintenance

MySQL 8

- Relationnel adapté aux transactions financières (ACID)
- Jointures complexes (agents ↔ kiosques ↔ transactions ↔ soldes)
- Déjà maîtrisé, hébergement facile (AlwaysData, VPS)

Blade + Tailwind + Alpine.js (pas React/Vue)

- Rendu serveur = SEO, simplicité, pas de SPA complexe
- Alpine.js pour interactivité légère (modales, toggles)
- Tailwind 4 = design system cohérent, responsive

Kotlin Android natif (pas Flutter)

- **Accès SMS obligatoire** : `BroadcastReceiver` , `Foreground Service`
- Fiabilité offline : `Room` (file d'attente) + `WorkManager` (sync)
- Les agents utilisent Android → pas besoin iOS
- Flutter aurait quand même nécessité du code natif Android pour les SMS

Docker

- Reproductibilité dev/prod
- Isolation MySQL + app + scheduler
- Déploiement VPS en une commande

MinIO

- Backups S3-compatible, séparés du VPS applicatif
- Résilience en cas de panne serveur principal

5. Sécurité — réponse complète au jury

5.1 Authentification web

- Guard Laravel `web` → modèle `Utilisateur`
- Mot de passe : **bcrypt** (champ `mot_de_passe`)
- **Changement obligatoire** à la 1ère connexion (`RequirePasswordChange`)
- Rate limiting login : **10 tentatives / minute** (`config/security.php`)
- Sessions stockées en BDD (`sessions`)

5.2 Contrôle d'accès — RBAC

- **Profils** (rôles) + **Liens** (routes/menus) + pivot `profil_liens`
- Middleware `CheckRoutePermission` sur toutes les routes protégées
- Résolution route → permission via `config/permissions.php`
- Matrice éditée dans l'UI (`PermissionController`)

Profils par défaut (`ProfilSeeder`) :

Profil	Niveau	Accès typique
Super Admin	0	Tout + sécurité + technique + RBAC
Admin	1	Gestion complète métier
Superviseur	2	Agents, kiosques, transactions, rapports
Comptable	2	Trésorerie, salaires, rapports
Agent	3	Dashboard agent uniquement

5.3 API mobile — deux tokens distincts

Token SMS API (envoi des transactions)

Étape	Détail
Génération	Une fois par l'admin web → <code>ConfigAppMobileController::generateToken()</code> → <code>Str::random(64)</code>

Stockage serveur	BDD config_app_mobile.api_token
Configuration mobile	Saisie dans l'app → AppPreferences (DataStore)
Envoi	ApiClient.kt ajoute Authorization: Bearer {token} à chaque requête SMS
Vérification	Middleware ValidateSmsApiToken → token invalide = 401

Ce token **n'est pas recréé à chaque envoi** : il est généré une fois, puis réutilisé.

Token agent login (espace agent)

Étape	Détail
Génération	À chaque login → MobileAgentController::login() → Str::random(64)
Stockage serveur	Cache Laravel 30 jours (mobile_agent:{token})
Usage	Dashboard, soldes, changement MDP, déconnexion
Ne sert pas à	Envoyer les transactions SMS auto

Phrase :

« Deux authentifications : le **token SMS API** autorise l'app à envoyer des transactions ; le **token agent login** identifie l'agent connecté pour son dashboard. Ce sont deux rôles différents. »

5.4 Sécurité des envois APK → web

Mesure	Détail
Bearer token	Secret partagé admin ↔ apps configurées
HTTPS en prod	URL API en https:// (obligatoire en production)
Validation données	Montant, type, agent, opérateur vérifiés côté serveur
Idempotence	Pas de doublon (référence opérateur ou hash SMS)
Audit	Tentatives token invalide journalisées

Vigilance :

- usesCleartextTraffic="true" dans le manifest → HTTP possible en dev seulement
- Token stocké en DataStore sur le téléphone → régénérable depuis le web si compromis

Phrase soutenance (30 sec) :

« Chaque envoi APK → API est authentifié par un Bearer token configuré côté admin, validé par middleware, avec données contrôlées et idempotence anti-doublon. En production, HTTPS est obligatoire. »

5.5 Rattachement agent : SIM + code agent (règle métier)

Les transactions SMS ne suivent pas l'agent connecté dans l'app — elles suivent la **SIM** et le **code agent du message**.

Règles

1. **SIM prime** — l'agent est identifié via le numéro SIM du téléphone (`SimUtils` → `AgentPhoneResolver`)
2. **Login agent ignoré** pour l'envoi SMS — pas d' `agent_id` de session
3. **Code agent dans le SMS** — si présent (`Code Agent: 5150328`), il **doit correspondre** au `code_agent` en base de l'agent de la SIM
4. **Incohérence** → transaction **refusée** (422)

Tableau des cas

SIM en base	Code dans SMS	Agent connecté app	Résultat
Agent A	5150328 (= code A)	Agent B ou aucun	✅ Transaction → Agent A
Agent A	9999999 (≠ code A)	n'importe	❌ Refusé — code ≠ SIM
SIM inconnue	5150328	n'importe	❌ Refusé — SIM non reconnue
Agent A	(absent)	Agent B	✅ Transaction → Agent A (SIM seule)

Fichiers clés

Fichier	Rôle
<code>AgentContextProvider.resolveForSmsTransaction()</code>	Envoie SIM + code SMS, pas session login
<code>AgentPhoneResolver::resolveForSmsTransaction()</code>	Résout agent par SIM + valide code
<code>SmsParser.extractCodeAgent()</code>	Extrait Code Agent: xxxxx du SMS Flooz/YAS

Prérequis données : le champ `agents.code_agent` doit contenir le **code opérateur** du SMS (ex. `5150328`), pas seulement `AG001` .

Phrase soutenance :

« La SIM identifie l'agent ; le code agent du SMS doit correspondre à celui en base. L'agent connecté dans l'app ne sert qu'au dashboard — pas au rattachement des transactions automatiques. »

5.6 Protection réseau & audit

- **Blocage IP automatique** après 5 échecs login (`IpBlockService` , `blocked_ips`)
- **Journal d'audit** : `system_logs` via trait `LogsActivity` (masquage champs sensibles)
- **Dashboard sécurité** : alertes, IP bloquées, résolutions
- **Backups chiffrés** vers MinIO (scheduler quotidien)
- Trust proxies pour Cloudflare/reverse proxy

5.7 Phrase clé pour le jury

« La sécurité est multicouche : authentification forte, RBAC granulaire route par route, tokens API pour le mobile, blocage IP automatique, audit complet des actions sensibles, et sauvegardes externalisées sur MinIO. »

6. RBAC vs ABAC — ce qu'il faut savoir

RBAC (Role-Based Access Control) — ce que vous utilisez

- L'accès dépend du **rôle** (Profil) de l'utilisateur
- Exemple : un Comptable accède à la trésorerie car son profil a le lien `gestion-entreprise.index`
- Implémentation : `profils` ↔ `profil_liens` ↔ `liens`

ABAC (Attribute-Based Access Control)

- L'accès dépend d'**attributs contextuels** : heure, IP, département, propriétaire de la ressource, etc.
- Exemple : « Un superviseur ne voit que les agents de **son** kiosque »
- Plus fin, plus complexe

Pourquoi RBAC et pas ABAC ?

RBAC (choisi)	ABAC (non retenu)
Simple à comprendre et administrer	Complexe à modéliser
Suffisant pour 5 profils métier	Overkill pour la taille actuelle
Matrice visuelle dans l'UI	Règles difficiles à expliquer au client
Standard pour ce type de SaaS	Utile si multi-tenant avancé

Évolution possible : ajouter des règles ABAC légères (ex. superviseur limité à ses kiosques) en complément du RBAC.

7. Scaling — montée en charge

État actuel (MVP / soutenance)

- 1 VPS app + 1 VPS MySQL + 1 VPS MinIO
- Cache/session/queue en **driver database**
- Pas de Redis, pas de réplication MySQL

Réponse au jury — plan de scaling

Court terme (100–500 utilisateurs)

1. **Cache Redis** : sessions, permissions, dashboard KPIs
2. **Queue Redis + workers** : exports PDF, envoi SMS, sync lourde
3. **Optimisation BDD** : index sur `transactions.created_at` , `agents.kiosque_id`
4. **CDN** pour assets statiques

Moyen terme (500–5000 utilisateurs)

5. **Load balancer** (Nginx/HAProxy) devant plusieurs instances Laravel
6. **MySQL read replica** : lectures dashboard/rapports sur replica
7. **Horizon** (supervision queues Laravel)

Long terme (SaaS multi-tenant — roadmap `docs/genuis.txt`)

8. **Multi-tenancy** : isolation par `entreprise_id` ou schémas séparés

- 9. **Sharding** par région/opérateur si volume SMS massif
- 10. **Auto-scaling** cloud (AWS/GCP) selon charge

Phrase clé

« L'architecture actuelle est monolithique modulaire, containerisée, prête à scaler horizontalement : on ajoute des instances app derrière un load balancer, on externalise cache/queue vers Redis, et on réplique MySQL en lecture. Le multi-tenant est prévu en roadmap. »

8. Impact du système d'exploitation

Sur le serveur (backend Laravel)

OS	Impact
Linux (Debian/Ubuntu)	Recommandé — Docker, Apache/Nginx, MySQL natifs
Windows	Possible (WAMP/XAMPP) mais moins adapté prod
macOS	Dev local uniquement

Notre choix : Linux sur VPS (Docker). L'app Laravel est **indépendante de l'OS** grâce à Docker.

Sur le client web

- **Aucun impact** : navigateur (Chrome, Firefox, Safari) — application web responsive
- Pas d'installation côté client

Sur l'app mobile

OS	Support
Android 8+ (API 26+)	Oui — app native Kotlin
iOS	Non — APIs SMS iOS très restreintes, pas de capture SMS opérateur

Phrase clé : « Le backend est containerisé donc OS-agnostic. Le mobile cible Android car l'interception SMS est une API Android exclusive. »

9. Organigramme — source et lien avec le système

Si le jury demande « Quelle est la source de l'organigramme ? »

L'organigramme fonctionnel du projet est dérivé de **l'analyse métier** du réseau d'agents Mobile Money :



Sources :

- 1. **L'organigramme de l'entreprise cliente** (structure réelle de son réseau d'agents)

2. **Ateliers de recueil des besoins** avec la direction et les responsables terrain
3. **Modélisation RBAC** implémentée dans `ProfilSeeder` et `DefaultProfilPermissions`
4. **Documentation métier** : `docs/genuis.txt`

Ce n'est **pas** un organigramme théorique — il reflète la **hiérarchie opérationnelle de l'entreprise** pour laquelle le produit a été livré.

Correspondance organigramme ↔ RBAC

Poste métier	Profil système
Directeur / DSI	Super Admin
Responsable réseau	Admin
Superviseur régional	Superviseur
Comptable	Comptable
Agent PDV	Agent

10. « Le ministère de l'Enseignement a changé ? »

Si c'est une question sur le gouvernement togolais

Oui, les structures gouvernementales évoluent régulièrement. Pour un document académique :

- Vérifier le **nom officiel actuel** sur gouv.tg ou le site de votre université/école
- Au Togo, le portefeuille enseignement supérieur relève historiquement du **MESR** (Ministère de l'Enseignement Supérieur et de la Recherche) — le nom exact peut avoir changé selon les remaniements gouvernementaux
- **Mettre à jour** la page de garde / remerciements si vous citez un ancien nom

Si c'est une question piège liée au projet

Non, le ministère n'a **aucun impact technique** sur PDV Connect. Ce projet est une solution **privée SaaS** pour entreprises de distribution Mobile Money, pas un système gouvernemental.

Réponse type : « Notre projet est indépendant de l'administration publique. Si la question porte sur notre établissement d'encadrement, nous nous référons au site officiel de l'université pour le nom à jour du ministère de tutelle. »

11. Questions jury — réponses prêtes

Q1 : Comment gérez-vous la sécurité sur la plateforme ?

Réponse (1 min) :

1. **Authentification** : sessions Laravel, bcrypt, changement MDP obligatoire, rate limiting
2. **Autorisation RBAC** : 5 profils, permissions route par route, matrice admin
3. **API mobile** : tokens Bearer distincts (SMS vs agent login)
4. **Réseau** : blocage IP auto, HTTPS en production, trust proxies
5. **Audit** : journal `system_logs` , dashboard sécurité
6. **Sauvegardes** : quotidiennes vers MinIO (VPS séparé)

7. **Données sensibles** : masquées dans les logs, jamais en clair

Q2 : Comment gérer le scaling pour la montée en charge ?

Réponse (1 min) :

1. Architecture **containerisée** → scale horizontal (plusieurs containers app)
 2. **Redis** pour cache/queue (prochaine étape)
 3. **Read replica MySQL** pour les rapports lourds
 4. **Load balancer** devant N instances
 5. **Multi-tenant** en roadmap pour isoler les clients SaaS
 6. App mobile : file offline Room → pas de surcharge serveur si réseau instable
-

Q3 : Le système d'exploitation a-t-il un impact ?

Réponse (30 sec) :

- **Serveur** : Linux + Docker → indépendant de l'OS hôte
 - **Client web** : aucun, navigateur suffit
 - **Mobile** : Android obligatoire (API SMS). iOS incompatible avec notre cas d'usage
-

Q4 : RBAC ou ABAC ?

Réponse (30 sec) :

- Nous utilisons **RBAC** : accès basé sur le profil (Admin, Agent...)
 - **ABAC** serait pertinent pour des règles contextuelles (ex. accès limité à son kiosque) — c'est une évolution possible
 - RBAC choisi pour simplicité, maintenabilité, et adéquation au nombre de rôles métier
-

Q5 : Pourquoi pas Flutter pour le mobile ?

Réponse (30 sec) :

- L'app doit intercepter les **SMS opérateurs** en arrière-plan
 - Ça nécessite les APIs Android natives (`BroadcastReceiver` , `Foreground Service`)
 - Flutter aurait requis du code natif Android quand même
 - Kotlin natif = fiabilité maximale + Room offline + WorkManager sync
-

Q6 : Comment fonctionne le flux SMS → transaction ?

Réponse (1 min) :

1. Opérateur envoie SMS au téléphone agent (YAS/FLOOZ)
 2. `SmsReceiver` Android capture le SMS (priorité 999)
 3. `SmsParser` extrait montant, type, référence, opérateur
 4. `AgentContextProvider` envoie **SIM + code agent SMS** (pas session login)
 5. `AgentPhoneResolver` valide SIM + cohérence code agent
 6. Si offline → queue Room ; sinon → `POST /api/transactions/from-sms` avec Bearer token
 7. Laravel `TransactionController::storeFromSms` crée la transaction + MAJ soldes
 8. Dashboard mis à jour en temps réel
-

Q7 : Quelles sont les limites actuelles / améliorations futures ?

Réponse honnête (force en soutenance) :

Limite actuelle	Amélioration prévue
Mono-entreprise	Multi-tenant SaaS
Cache/queue en BDD	Redis
Token agent en cache (pas JWT)	Sanctum/JWT
HTTP autorisé sur mobile (dev)	HTTPS strict en prod
Pas de 2FA	Authentification à deux facteurs

12. Schéma flux transaction SMS (à dessiner au tableau)



13. Checklist avant soutenance

- ☐ Corriger « Flutter » → « Kotlin Android natif » dans le mémoire
- ☐ Vérifier nom du ministère de tutelle (site officiel)
- ☐ Répéter le scénario démo (section 15) **3 fois**
- ☐ Préparer captures d'écran de secours si internet tombe

- ☐ Avoir l'APK ou Appetize.io prêt
- ☐ Connaître les 5 profils RBAC et leurs accès
- ☐ Savoir expliquer architecture (3 tiers, MVC, REST) et méthode d'analyse (section 2bis)
- ☐ Savoir expliquer RBAC vs ABAC en 30 secondes
- ☐ Maîtriser sécurité APK → web + 2 tokens + règle SIM/code agent (section 5.3–5.5)
- ☐ Maîtriser les 3 difficultés (section 17)
- ☐ Savoir expliquer Room, WorkManager et détection réseau (section 18)
- ☐ Connaître les opérateurs : YAS (Mixx), FLOOZ (Moov Money)
- ☐ Lire sections 14 (plan oral), 21 (modules métier), 22 (formats SMS)
- ☐ Maîtriser section 19 (contribution en équipe — backend + mobile pour toi)

14. Plan de présentation oral (15–20 min)

Structure recommandée

Min	Partie	Contenu	Support
0–2	Introduction	Qui tu es, contexte entreprise cliente , produit livré en production	Oral
2–4	Problématique	Saisie manuelle, erreurs, retards, pas de visibilité réseau agents/kiosques	Oral (+ slide problème)
4–6	Solution PDV Connect	Web + mobile SMS auto + centralisation	Schéma architecture §2
6–11	Démo live	Login → dashboard → transaction SMS → RBAC	Section 15
11–14	Aspects techniques	Architecture 3 tiers, BDD, sécurité (tokens, SIM, RBAC), Agile	§2bis, §3, §5
14–16	Difficultés & résolutions	Parsing SMS, identification agent SIM, offline Room	Section 17
16–18	Limites & perspectives	Mono-entreprise, Redis, HTTPS strict, multi-tenant	Section Q7
18–20	Conclusion	Récap valeur client + remerciements	Oral

Problématique métier (2 min — à apprendre)

« L'entreprise cliente gérait un réseau d'agents Mobile Money au Togo (YAS/Mixx, FLOOZ). Chaque transaction générait un **SMS opérateur**. Les agents devaient **saisir manuellement** montant, type, client

— source d'**erreurs**, de **retards** et d'**écarts** entre le solde opérateur et le solde enregistré. Les superviseurs n'avaient **pas de visibilité en temps réel** sur les kiosques, la trésorerie et la paie. PDV Connect automatise cette chaîne. »

Phrase d'ouverture (30 sec)

« Bonjour. Nous présentons **PDV Connect**, solution commandée par une entreprise cliente, déjà **livrée et déployée**, pour digitaliser la gestion des agents Mobile Money : capture automatique des SMS, centralisation web, contrôle d'accès et pilotage trésorerie/paie. »

Phrase de clôture (20 sec)

« PDV Connect réduit la saisie manuelle, fiabilise les transactions via la SIM et le parsing SMS, et donne à l'entreprise une vision temps réel de son réseau. Merci — nous sommes prêts pour vos questions. »

Correction Flutter dans le mémoire (si le jury voit le document)

« Une erreur de vocabulaire s'est glissée dans le mémoire : nous avons écrit Flutter, alors que l'application mobile est développée en **Kotlin Android natif**, ce qui est nécessaire pour intercepter les SMS opérateurs. Le mémoire a été corrigé. »

15. Scénario démo (5–7 min) — à répéter 3 fois

Avant de commencer

- Lancer Docker : `./docker-start.sh` ou `docker compose up -d`
- Vérifier : `http://localhost` (ou ton URL VPS)
- Onglets ouverts à l'avance :
 1. Login
 2. Dashboard
 3. Transactions
 4. Rôles & permissions
 5. (Optionnel) App mobile Appetize ou téléphone

Compte démo : `admin@pdvconnect.com` / `password123`

→ Si changement MDP demandé : en profiter pour montrer la **sécurité 1ère connexion**.

Étape 1 — Contexte (30 sec, sans écran)

« Un agent PDV reçoit des SMS YAS ou FLOOZ à chaque transaction. Avant PDV Connect, il fallait tout saisir à la main. Maintenant l'app Android lit le SMS et l'envoie automatiquement à la plateforme. »

Étape 2 — Login + Dashboard (1 min 30)

1. Se connecter en Admin
 2. Montrer le **dashboard** :
 - KPIs du mois (CA, nb transactions)
 - Graphique par opérateur (YAS / FLOOZ)
 - Carte des kiosques / zones
 3. **Phrase clé** : « Tout est centralisé en temps réel pour le superviseur. »
-

Étape 3 — Transaction SMS (2 min) ★ cœur du projet

Option A — Avec émulateur Android / téléphone :

```
cd android-app
./scripts/send-test-sms.sh flooz-retrait
```

→ Rafraîchir la liste transactions sur le web.

Option B — Sans mobile (secours) :

- Montrer une transaction déjà en base (source = SMS)
- Ouvrir le détail : montant, opérateur, référence, agent

Ce qu'il faut dire en montrant :

« Le SMS arrive sur le téléphone → `SmsParser` extrait montant et type → API Laravel crée la transaction → les soldes agent sont mis à jour → le dashboard se met à jour. »

Étape 4 — RBAC en 1 min

1. Aller dans **Rôles & permissions** → **Gestion permissions**
2. Montrer la matrice profils x menus
3. **Phrase clé** : « Un Comptable voit trésorerie et salaires. Un Agent ne voit que son dashboard. C'est du RBAC route par route. »

(Optionnel : se déconnecter, se connecter en Comptable si compte dispo)

Étape 5 — Mobile ou clôture (1 min)

Avec Appetize / APK :

- Montrer l'écran config (URL API + token)
- Montrer l'espace agent (soldes YAS/FLOOZ)

Sans mobile :

« L'APK est distribué via `/app-mobile`. Version 2.9, Kotlin natif, file offline Room si pas de réseau. »

Étape 6 — Conclusion démo (30 sec)

« En résumé : capture SMS automatique, centralisation web, contrôle d'accès par profil, déploiement Docker sur VPS avec backups MinIO. »

Plan B si ça casse

Problème	Solution
Site inaccessible	Captures d'écran préparées + expliquer l'architecture
Login échoue	Vérifier Docker / php artisan migrate --seed
Pas de mobile	Montrer transaction SMS existante + schéma au tableau
Dashboard vide	Lancer DemoDataSeeder ou montrer données de test

16. Questions pièges — réponses courtes

Technique

Question	Réponse
Pourquoi Laravel et pas Node/Django ?	ORM mature, écosystème PHP accessible au Togo, rapidité de dev, hébergement simple
Normalisation BDD ?	3NF — transactions séparées des soldes, pivots RBAC, pas de redondance
SMS en double ?	Idempotence par référence opérateur (Txn ID) ou hash SHA1 du SMS brut + agent
Pas de réseau mobile ?	Queue Room locale → WorkManager resynchronise au retour réseau (voir section 18)
C'est quoi Room / WorkManager ?	Room = BDD SQLite locale ; WorkManager = tâches Android en arrière-plan
Comment l'app sait s'il y a du réseau ?	NetworkUtils (vérif instantanée) + NetworkMonitor (écoute continue)
Pourquoi pas Redis pour la file ?	Redis = serveur ; notre problème = coupure réseau avant l'envoi → file sur le téléphone
Pourquoi pas Flutter ?	APIs SMS Android natives obligatoires — Kotlin plus fiable pour ce cas
Comment tester le parser SMS ?	Tests unitaires SmsParserTest.kt + SmsMessageAssemblerTest.kt
Que se passe-t-il si le format SMS change ?	Mise à jour des regex dans SmsParser.kt — formats documentés en commentaires
Les envois APK → web sont-ils sécurisés ?	Bearer token + HTTPS prod + validation serveur + idempotence (section 5.4)
C'est quoi le Bearer token ?	Secret généré une fois par l'admin, ajouté à chaque requête — pas recréé à chaque envoi (section 5.3)
Token SMS vs token agent login ?	Token SMS = autoriser envoi transactions ; token login = session dashboard agent (section 5.3)
Agent connecté ≠ agent de la transaction ?	Transaction SMS → agent de la SIM , pas l'agent connecté (section 5.5)
Code agent dans le SMS doit correspondre ?	Oui — doit matcher agents.code_agent de l'agent SIM, sinon 422 (section 5.5)
Agent X connecté sur SIM de Y ?	SMS crédités à l'agent propriétaire de la SIM ; code agent du SMS validé (section 5.5)

Métier / jury

Question	Réponse
Qui a commandé le produit ?	Une entreprise cliente (réseau d'agents Mobile Money au Togo)
Le produit est-il en production ?	Oui — plateforme web sur VPS, APK v2.9 chez les agents
Qui paie pour la solution ?	L' entreprise cliente (prestation / licence selon contrat)
Combien coûte le déploiement ?	~2 VPS (10–20 €/mois chacun) + domaine + MinIO
Et si l'opérateur change le SMS ?	Parser modulaire par format (Mix, Flooz, apport virtuel) — maintenance prévue
Vos données sont où ?	MySQL sur VPS 1, backups chiffrés sur MinIO VPS 2
C'est sécurisé pour de l'argent ?	RBAC, audit, tokens API, idempotence — pas un core banking, outil de gestion réseau
Quelle est VOTRE contribution ?	(À personnaliser — voir section 18)

Académique

Question	Réponse
Méthode utilisée ?	Agile (Scrum) : sprints, backlog, livraisons incrémentales (section 2bis)
Quelle architecture ?	Client/serveur 3 tiers + MVC Laravel + couches Android + API REST (section 2bis)
Pourquoi ce sujet ?	Répondre au besoin réel d'une entreprise cliente : digitaliser la gestion de son réseau d'agents Mobile Money
Limites du projet ?	Mono-entreprise, pas de 2FA, tests partiels — voir section Q7

17. Difficultés rencontrées + résolutions (à dire à l'oral)

Ces difficultés viennent de l'historique réel du projet (commits git). Adapte la formulation « je / nous » selon ton équipe.

Difficulté 1 — Parsing SMS hétérogènes et multipart

Le problème

Les opérateurs YAS (Mixx) et FLOOZ (Moov) n'envoient **pas le même format** de SMS. Exemples réels dans le code :

- Flooz retrait : Txn ID 040228895463... Montant: 2 000,00 FCFA...

- Mix retrait : retrait de 4 900 FCFA effectuée par 91069102...
- Mix apport virtuel : L'agent 10019 (NOM) vous a envoyé 50 000 FCFA...

Autre problème : les SMS longs arrivent en **plusieurs morceaux** (multipart UCS-2 avec accents). L'app Messages les affiche fusionnés, mais Android livre des fragments séparés → parsing sur un morceau = montant ou type incorrect.

Impact

- Transactions non créées ou montants erronés
- Doublons si le même SMS est retraité après un échec réseau

Comment on a résolu

1. **SmsMessageAssembler.kt** — recolle les segments multipart avant parsing
2. **SmsParser.kt** — chaîne de parseurs spécialisés (apport virtuel → envoi → Mix → Flooz → fallback générique) + normalisation texte (espaces insécables, apostrophes)
3. **Tests unitaires** — `SmsParserTest.kt`, `SmsMessageAssemblerTest.kt` avec SMS réels terrain
4. **Idempotence côté API** — dans `TransactionController::storeFromSms` : clé stable `SMS-{sha1(agent + sms normalisé)}` si pas de référence opérateur → pas de doublon en base

Phrase soutenance (45 sec)

« Notre première difficulté majeure était la fiabilité du parsing SMS. YAS et FLOOZ ont des formats différents, et Android fragmente les longs SMS. On a créé un assembleur multipart, un parser modulaire par format opérateur, des tests unitaires sur des SMS réels, et une idempotence côté serveur pour éviter les doublons. L'APK est passé de v1.7 à v2.9 avec ces corrections. »

Difficulté 2 — Identifier le bon agent à partir du SMS

Le problème

Quand un SMS arrive sur le téléphone, la plateforme doit savoir **à quel agent** rattacher la transaction. Or :

- Le SMS ne contient pas toujours le même identifiant (code agent YAS 5150328, numéro client 79984409, parfois rien de fiable)
- En base, le téléphone agent peut être enregistré avec ou sans indicatif (+228, 228, 8 chiffres seulement)
- Un agent peut se connecter dans l'app sur un téléphone qui n'est **pas** le sien → risque d'affectation au mauvais compte

Résultat en tests terrain : transactions rejetées avec « Agent introuvable » ou affectées au mauvais compte.

Impact

- SMS bien parsés mais **non enregistrés** côté serveur
- Plusieurs commits de correction (correction agent number v1 → v5)

Règle métier retenue : la transaction est rattachée à l'agent **propriétaire de la SIM** qui a reçu le SMS. Si le SMS contient un **code agent** (Code Agent: 5150328), il **doit correspondre** au `code_agent` de cet agent en base — sinon la transaction est **refusée** (422).

Comment on a résolu

1. **Côté mobile** — `AgentContextProvider.resolveForSmsTransaction()`
 - Envoie le numéro **SIM** (`SimUtils`) — pas l'`agent_id` de session login

- Transmet le **code agent extrait du SMS** pour validation serveur

2. Côté serveur — `AgentPhoneResolver::resolveForSmsTransaction()`

- Résout l'agent via la **SIM**
- Si code agent présent dans le SMS → **doit matcher** `agents.code_agent`
- Incohérence → erreur 422, transaction non enregistrée

3. `AgentPhoneResolver.php` : téléphone (8 derniers chiffres) + comparaison de codes

Phrase soutenance (45 sec)

« Un SMS est rattaché à l'agent **dont la SIM a reçu le message**. Si le SMS indique un code agent — par exemple `Code Agent: 5150328` — il **doit correspondre** au code enregistré pour cet agent. Sinon on refuse la transaction : c'est une cohérence métier évidente entre le téléphone physique et le message opérateur. »

Difficulté 3 — Fiabilité réseau : ne jamais perdre un SMS

Détails techniques Room / WorkManager / réseau → **section 18**

Le problème

Les agents travaillent souvent avec un **réseau instable** (3G faible, coupures). Si l'app envoie le SMS à l'API et que la requête échoue :

- La transaction est perdue
- Ou elle est renvoyée plusieurs fois → doublons

Impact

- Perte de confiance des agents (« l'app ne marche pas »)
- Écart entre le solde réel opérateur et le solde dans PDV Connect

Comment on a résolu

1. `OfflineSyncRepository.kt` — chaque SMS parsé entre d'abord dans une **file Room** (`PendingTransactionEntity`) avant envoi
2. **États de la file** : pending → syncing → synced / failed — reprise possible après crash
3. `TransactionSyncWorker (WorkManager)` — resynchronisation automatique au retour réseau, même app fermée
4. `NetworkMonitor` — déclenche la sync dès que la connexion revient
5. **Dédoublonnage local** — `findByReference()` avant insertion + idempotence serveur (difficulté 1)
6. **Réparation file corrompue** — `commit fix(mobile):` réparation des entrées en file héritées (APK v2.1)

Phrase soutenance (45 sec)

« Au Togo le réseau coupe souvent. On ne peut pas envoyer le SMS directement à l'API et espérer que ça passe. L'app enregistre d'abord chaque transaction en local dans Room, puis WorkManager retente l'envoi automatiquement. Ainsi, aucun SMS n'est perdu : il part dès que le réseau revient, sans action de l'agent. »

Difficulté 4 (optionnelle si le jury en demande une 3^e) — Mise à jour obligatoire de l'APK

Le problème

Quand on corrige le parser SMS (v1.7 → v2.9), les agents en terrain gardent l'ancienne version → nouveaux formats non reconnus.

Résolution

- `MobileApkVersion.php` + fichier `pdv-connect.version.json` généré au build
- `AppUpdateChecker.kt` compare `minVersionCode` et bloque l'app si trop ancienne
- Page publique `/app-mobile` pour télécharger la dernière APK

Phrase (20 sec)

« Chaque correction du parser exige une MAJ côté agents. On vérifie la version au lancement et on force la mise à jour si elle est inférieure au minimum requis. »

18. Mobile offline-first : Room, WorkManager, détection réseau

Section à maîtriser pour expliquer la difficulté 3 et répondre aux questions techniques mobile.

18.1 Room — c'est quoi ?

Room = bibliothèque Android (Google) pour une **mini base de données SQLite sur le téléphone**.

Elle stocke les SMS/transactions **en attente d'envoi** quand le réseau est coupé.

Analogie : un carnet de brouillons sur le téléphone. Tant que le message n'est pas parti au serveur, il reste écrit localement.

Dans le projet :

Fichier	Rôle
<code>AppDatabase.kt</code>	Base SQLite locale
<code>PendingTransactionEntity.kt</code>	Structure d'une transaction en attente
<code>PendingTransactionDao.kt</code>	Requêtes : insert, update, findByReference, getPending

Exemple de file locale :

id	montant	type	statut
1	2000	retrait	pending
2	5000	depot	synced

18.2 WorkManager — c'est quoi ?

WorkManager = planificateur Android qui **exécute des tâches en arrière-plan**, même si l'app est fermée.

Chez PDV Connect, il relance l'envoi des transactions stockées dans Room.

Analogie : un employé qui retente d'envoyer les brouillons dès qu'Internet revient, sans que l'agent fasse quoi que ce soit.

Dans le projet :

Fichier	Rôle
TransactionSyncWorker.kt	Lit Room et envoie les SMS en attente à l'API
SyncScheduler.kt	Planifie la sync (immédiate + toutes les 15 min)
OfflineSyncRepository.kt	Logique : enqueue → sync → états

Contrainte WorkManager : la tâche ne part que si `NetworkType.CONNECTED` (réseau disponible).

18.3 Qui gère la file ? (mobile vs serveur)

Deux « queues » distinctes dans le projet :

A. File offline mobile (SMS) — la principale

Couche	Techno	Rôle
Stockage	Room (SQLite)	Table pending_transactions
Accès	PendingTransactionDao	CRUD sur la file
Logique	OfflineSyncRepository	Enqueue → tente envoi → gère états
Exécution	WorkManager	TransactionSyncWorker
Planification	SyncScheduler	Sync immédiate + périodique (15 min)
Déclencheur	NetworkMonitor	Sync dès retour réseau

États de la file : `pending` → `syncing` → `synced` / `failed`

B. Queue Laravel (serveur) — secondaire

- Driver : `database` (`config/queue.php` → table `jobs`)
- Pour tâches serveur (exports, etc.)
- **Pas utilisée** pour le flux SMS mobile

Phrase clé soutenance :

« La file est gérée par **Room** en local sur le téléphone, et **WorkManager** relance l'envoi automatiquement quand le réseau revient. Ce n'est pas une queue Redis côté serveur — c'est une file **offline-first** sur l'app Android. »

Offline-first = on enregistre **d'abord en local**, on envoie **ensuite**. Rien n'est perdu si le réseau coupe.

18.4 Comment l'app sait s'il y a du réseau ?

Deux mécanismes complémentaires :

1. Vérification instantanée — `NetworkUtils.isOnline()`

À chaque SMS ou action, l'app interroge Android via `ConnectivityManager` :

- Wi-Fi, 4G/3G, Ethernet ou VPN
- Capacité **INTERNET** (pas seulement « connecté au Wi-Fi » sans data)

Fichier : `android-app/.../util/NetworkUtils.kt`

Utilisé quand :

- SMS reçu → online : envoi direct ; offline : Room + attente
- Login agent, changement MDP, vérification MAJ (MainActivity)

2. Écoute continue — NetworkMonitor

Au démarrage de l'app (`PdvConnectApp.onCreate()`), Android enregistre un **écouteur réseau** :

Réseau revient → `onAvailable()` → `SyncScheduler` → `WorkManager` → envoi Room

Fichier : `android-app/.../sync/NetworkMonitor.kt`

L'agent n'a **rien à faire** : la sync part toute seule.

18.5 Flux complet (SMS → serveur)

```

SMS opérateur reçu (SmsReceiver)
    ↓
SmsMessageAssembler (recolle multipart)
    ↓
SmsParser (extrait montant, type, référence)
    ↓
OfflineSyncRepository.enqueueSmsTransaction()
    ↓
Room – sauvegarde locale (TOUJOURS, même sans réseau)
    ↓
NetworkUtils.isOnline() ?
    |
    ├── OUI → syncAll() → POST /api/transactions/from-sms
    |                               → statut synced
    |
    └── NON → WorkManager en attente (contrainte réseau)
            + NetworkMonitor écoute...
              ↓
Réseau revient → TransactionSyncWorker
              ↓
POST /api/transactions/from-sms → synced

```

Démarrage app (`PdvConnectApp.kt`) :

- `NetworkMonitor.register()` — écoute réseau
- `SyncScheduler.schedulePeriodic()` — sync toutes les 15 min

18.6 Questions jury — Room / WorkManager / réseau

Question	Réponse
C'est quoi Room ?	Mini BDD SQLite sur le téléphone pour stocker les SMS en attente
C'est quoi WorkManager ?	Planificateur Android qui relance l'envoi en arrière-plan

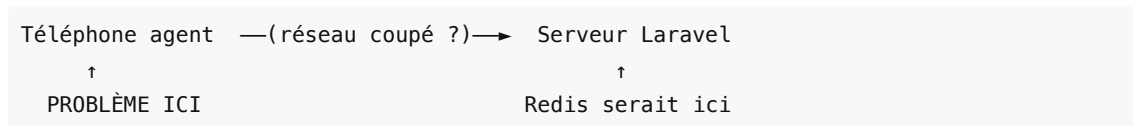
Pourquoi pas Redis ?	Redis = file serveur . Notre problème = coupure réseau avant l'envoi → il faut stocker sur le téléphone
Comment l'app détecte le réseau ?	API Android <code>ConnectivityManager</code> via <code>NetworkUtils</code> + écouteur <code>NetworkMonitor</code>
Et si l'app est fermée ?	<code>WorkManager</code> continue — Android relance la sync au retour réseau
Doublons si renvoi ?	<code>findByReference()</code> en local + idempotence SHA1 côté API (section 17)
Queue Laravel alors ?	Table <code>jobs</code> pour tâches serveur, pas pour le flux SMS mobile

Phrase si on demande « comment Android le sait ? » :

« Ce n'est pas notre code qui ping Google. On utilise l'**API native Android** `ConnectivityManager` : le système informe l'app en temps réel des changements Wi-Fi / 4G. »

18.7 Pourquoi pas Redis pour la file ?

Parce que le problème n'est pas au même endroit.



Redis gère des tâches **déjà arrivées sur le serveur**.

Chez PDV Connect, le SMS peut **ne jamais atteindre le serveur** si la 4G coupe au moment de l'envoi.

Room (mobile) vs Redis (serveur)

Situation	Room + WorkManager	Redis
Pas de réseau au moment du SMS	✅ Stocke en local	❌ Serveur jamais contacté
App fermée, réseau revient	✅ WorkManager renvoie	❌ Rien à renvoyer
Agent en zone mal couverte	✅ Offline-first	❌ Inutile à ce stade

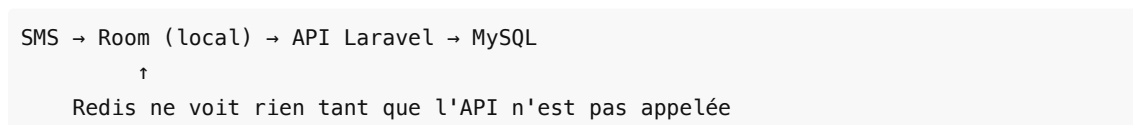
Redis ne peut pas stocker ce que le téléphone n'a pas encore envoyé.

Et Redis côté Laravel alors ?

Redis **serait pertinent** pour d'autres usages serveur :

- Exports PDF / Excel lourds
- Envoi d'emails
- Scaling avec plusieurs workers Laravel (Horizon)

Mais **pas pour remplacer Room** sur le flux SMS :



Actuellement le projet utilise la queue Laravel en `database` (`config/queue.php` → table `jobs`) — suffisant pour le MVP.

Phrase soutenance (30 sec)

« Redis est une file **serveur** pour traiter des jobs déjà reçus. Notre contrainte terrain, c'est la coupure réseau **avant** l'envoi. On a donc une file **offline-first** sur le téléphone avec Room, et WorkManager renvoie dès que la 4G revient. Redis reste une évolution possible côté Laravel pour les exports et le scaling, pas pour remplacer Room. »

Si le jury insiste : « Et les deux ? »

« Oui — **Room** pour la fiabilité mobile, **Redis** plus tard côté serveur pour les tâches lourdes et la montée en charge. Ce sont deux niveaux complémentaires, pas substituables. »

19. Ta contribution personnelle

Question quasi systématique — préparer une réponse claire en **30 secondes**.

Questions possibles : « Qu'avez-vous fait vous-même ? » · « Quelle était votre part ? » · « Qui a fait quoi ? »

Règle oral : parler en « nous / notre équipe »

Utiliser « **nous** » pour le projet global, puis « **de mon côté** » pour ta part. Ne jamais dire « j'ai tout fait seul » — présenter une **répartition d'équipe** cohérente.

Répartition équipe (à connaître par cœur)

Personne	Domaine principal	Modules / fichiers
Toi (présentateur)	Backend + Mobile	API SMS, TransactionController, RBAC, SmsParser, sync offline (Room/WorkManager), identification SIM/code agent
Coéquipier A	Frontend web	Dashboards Blade, rapports PDF/Excel, interface agents/kiosques, gestion permissions (UI)
Coéquipier B	BDD + Infra	Migrations, seeders, Docker VPS, MinIO backups, déploiement production

(Adapter les prénoms si le jury demande — rester vague : « mon coéquipier sur le front web ».)

Ta part — tableau détaillé

Domaine	Ma contribution
Backend Laravel	API SMS (/api/transactions/from-sms), TransactionController, AgentPhoneResolver, idempotence, middleware token
Mobile Android	SmsParser, SmsReceiver, OfflineSyncRepository, AgentContextProvider, espace agent

Intégration	Chaîne SMS → API → soldes ; tests SmsParserTest, AgentPhoneResolverTest
Frontend web	(Coéquipier A — ne pas s'étendre, renvoyer vers lui)
Infra / BDD	(Coéquipier B — idem)

Phrase principale (30 sec) — à apprendre

« PDV Connect a été développé **en équipe** sur commande d'une entreprise cliente. **De mon côté**, j'ai surtout travaillé sur le **backend Laravel** — l'API SMS, la validation des transactions et le RBAC — et sur l'**application Android** : parsing des SMS YAS/FLOOZ, file offline Room et synchronisation. Mes coéquipiers ont pris en charge l'**interface web** et l'**infrastructure** Docker. Le produit est livré et en production. »

Si on te demande « Qui a fait quoi ? »

« Nous avons découpé par couche : **moi** le cœur métier mobile et l'API, **[coéquipier A]** les écrans web et les rapports, **[coéquipier B]** la base de données et le déploiement. On a fait des **revues croisées** à chaque sprint pour valider l'intégration. »

Si on insiste : « Montrez quelque chose que vous avez codé »

Montrer en priorité (tu maîtrises tout ça) :

1. `SmsParser.kt` ou flux SMS dans la démo
2. `TransactionController::storeFromSms` ou `AgentPhoneResolver`
3. `OfflineSyncRepository` si question offline

« Voici la partie que j'ai implémentée : le parser SMS et son envoi vers l'API, avec validation SIM et code agent côté serveur. »

Formulations à éviter

 Ne pas dire	 Dire plutôt
« J'ai tout fait seul »	« Notre équipe a livré le projet en répartissant les modules »
« Il n'y avait personne d'autre »	« Chacun avait son périmètre, avec des points de sync réguliers »
« Je connais tout le code parce que... »	« J'ai surtout travaillé sur le backend et le mobile, mais je connais l'architecture globale »

Cohérence avec le reste du doc

- Section **14** : utiliser « **nous** présentons PDV Connect »
- Section **2bis Agile** : « **nos** sprints », « **notre** backlog »
- Démo : « **Notre** application intercepte le SMS... »

20. Comptes de démonstration

Compte	Rôle	Remarque
admin@pdvconnect.com / password123	Super Admin	Changement MDP obligatoire à la 1ère connexion
Agents demo AG001–AG005	Agent	Via DemoDataSeeder

21. Modules métier web + logique des soldes

Modules principaux (à présenter à l'oral)

Module	Rôle	Profils concernés
Dashboard	KPIs temps réel, graphiques, carte zones/kiosques	Admin, Superviseur
Transactions	Liste, détail, saisie manuelle, annulation, export	Admin, Superviseur
Agents	CRUD agents, soldes, affectation kiosque	Admin, Superviseur
Kiosques	Points de vente, carte interactive, géolocalisation	Admin, Superviseur
Opérations agence	Approvisionnements, montants initiaux (hors SMS auto)	Admin, Superviseur
Gestion entreprise	Trésorerie + Paie/salaires + paramètres commission	Admin, Comptable
Rapports	Statistiques, top agents, exports PDF/Excel	Admin, Superviseur, Comptable
RBAC	Rôles, permissions, routes/menus	Super Admin
Paramètres app mobile	Token API SMS, filtres opérateurs, distribution APK	Super Admin
Sécurité / Technique	IP bloquées, logs, backups	Super Admin
Espace agent (mobile)	Dashboard agent, soldes YAS/FLOOZ, historique	Agent terrain

Valeur pour l'entreprise cliente

- **Gain de temps** : plus de saisie manuelle des SMS
- **Fiabilité** : moins d'erreurs de montant ou de doublons (idempotence)
- **Pilotage** : superviseur voit tout le réseau en temps réel
- **Paie automatisée** : commissions calculées depuis les transactions

Logique des soldes agent

Chaque agent a **deux types de soldes** :

Type	Signification
Espèce	Cash en caisse chez l'agent
Virtuel	Solde sur le réseau opérateur (YAS ou FLOOZ), un par opérateur

À chaque transaction SMS validée (`updateAgentBalance`)

Solde virtuel (opérateur) :

- Si le SMS contient « nouveau solde » → on prend cette valeur (`virtual_balance_after`)
- Sinon → ancien solde virtuel + montant transaction
- Nouvelle ligne dans table `soldes` (historique, pas écrasement)

Solde espèce (caisse) — dépôt ou retrait :

- Espèce diminue du montant (l'agent mobilise du cash pour le réseau)
- Plancher à 0 (`max(0, ...)`)

Apport virtuel (cas particulier Mix)

SMS type : « *L'agent 10019 (NOM) vous a envoyé 50 000 FCFA...* »

- Catégorie `apport_virtuel` — crédit **float Mix** depuis une agence
- **Pas de mouvement espèce** — seulement virtuel YAS/Mixx
- Parser : `SmsParser.parseApportVirtuel()`

Annulation transaction

- Règle : possible sous **24 h**, transaction `valide`, pas opération agence
- **Inverse les soldes** avant de passer le statut à `annule`
- Mobile agent : token session + ownership vérifiés

Phrase soutenance (30 sec) :

« Chaque transaction met à jour deux soldes : l'**espèce** en caisse et le **virtuel** chez l'opérateur. Quand le SMS indique le nouveau solde opérateur, on l'utilise directement — sinon on calcule. L'apport virtuel ne touche que le float Mix. L'annulation inverse les mouvements dans la limite de 24 heures. »

22. Formats SMS opérateurs (Mixx / Flooz)

Source terrain : `message_type.txt` — formats réels utilisés par `SmsParser.kt`.

YAS / Mixx by Yas

Retrait

```
retrait de 4 900 FCFA effectue par 91069102 le 01-11-25 12:36
commission : 21 FCFA votre nouveau solde Mixx : 287734 FCFA (commission incluse) REF
13990966494
```

Champ extrait	Valeur exemple
Type	retrait
Montant	4 900 FCFA

Commission	21 FCFA
Référence	13990966494
Solde virtuel après	287 734 FCFA
Opérateur	YAS

Dépôt

Depot de 2 000 FCFA effectuee pour 90513298(AHIAKPOR) le 01-11-25 12:45
Commission: 14 FCFA nouveau solde mix : 275 784 (commission incluse) REF 13990436494

Champ extrait	Valeur exemple
Type	depot
Montant	2 000 FCFA
Client	90513298 (AHIAKPOR)
Commission	14 FCFA
Référence	13990436494

Apport virtuel (agence → agent)

L'agent 10019 (SPT NYEKONAKPOE) vous a envoyé 50 000 FCFA...

Champ extrait	Valeur
Catégorie	apport_virtuel
Agent source	10019
Montant	50 000 FCFA

FLOOZ / Moov Money

Retrait

Txn ID 040228895463 07/02/2025
retrait valider par le client FABIO ,79984409
Veillez remettre l'argent au client
Montant: 2 000,00 FCFA Commission Net : 24,32 FCFA
Code Agent: 5150328
Date:07/02/2025 12:47:22. Nouveau solde FLOOZ : 28 703,00 FCFA.

Champ extrait	Valeur exemple
Type	retrait

Montant	2 000 FCFA
Code agent	5150328 ← validé vs SIM
Référence	Txn ID 040228895463
Solde Flooz après	28 703 FCFA
Opérateur	FLOOZ

Dépôt

Depot reussi Montant:2600,00 FCFA beneficiaire : 96096844 Date : 01/11/2025 14:13:04
Commission Net : 13,44 FCFA Nouveau solde : 26 103,00 FCFA Txn ID: 040229031027

Champ extrait	Valeur exemple
Type	depot
Montant	2 600 FCFA
Bénéficiaire	96096844
Référence	040229031027

Test SMS en démo (émulateur)

```
cd android-app
./scripts/send-test-sms.sh flooz-retrait # Flooz + Code Agent
./scripts/send-test-sms.sh mix-retrait  # Mix retrait
./scripts/send-test-sms.sh all          # 4 SMS de test
```

Phrase soutenance :

« YAS et FLOOZ n'utilisent pas le même format. Notre parser est modulaire : Mix retrait/dépôt/envoi, Flooz avec Code Agent, et apport virtuel agence. Chaque format est testé unitairement dans `SmsParserTest.kt` . »

Document de préparation soutenance — PDV Connect, produit commandé par une entreprise cliente, livré en production.